ADR-004-workflow-execution-model

ADR-004: Workflow Execution Model

Status

Accepted

Date

2026-01-08

Context

HelixCode provides automated development workflows that orchestrate multiple steps including code analysis, generation, testing, building, and deployment. The workflow system needs to support:

1. **Multiple workflow types**: Planning, building, testing, refactoring, debugging
2. **Step dependencies**: Steps may depend on other steps completing first
3. **LLM integration**: Steps can use AI for analysis and generation
4. **Concurrent execution**: Independent steps should execute in parallel
5. **Error handling**: Graceful handling of step failures
6. **Security**: Command execution must be safe from injection attacks
7. **Multi-language support**: Support Go, Python, Node.js, Rust, and more
8. **Metrics collection**: Track execution time, LLM usage, and outcomes
9. **Autonomy modes**: Support different levels of automation

The challenge was designing a flexible yet secure execution model that can leverage AI capabilities while maintaining deterministic, auditable execution.

Decision

We implemented a DAG-based (Directed Acyclic Graph) workflow execution model with distinct step types and actions, combined with LLM integration for intelligent code generation and analysis.

Workflow Structure

```
type Workflow struct {
    ID          string         `json:"id"`
    Name        string         `json:"name"`
    Description string         `json:"description"`
    Mode        string         `json:"mode"`
    Steps       []Step         `json:"steps"`
    Status      WorkflowStatus `json:"status"`
    CreatedAt   time.Time      `json:"created_at"`
```

```
    UpdatedAt   time.Time      `json:"updated_at"`
}

type Step struct {
    ID           string     `json:"id"`
    Name         string     `json:"name"`
    Description  string     `json:"description"`
    Type         StepType   `json:"type"`
    Action       StepAction `json:"action"`
    Dependencies []string   `json:"dependencies"`
    Status       StepStatus `json:"status"`
    Error        string     `json:"error,omitempty"`
}
```

Step Types

Four fundamental step types:

1. **Analysis** (`StepTypeAnalysis`): Examine code, requirements, or context
2. **Generation** (`StepTypeGeneration`): Create new code or content
3. **Execution** (`StepTypeExecution`): Run commands or scripts
4. **Validation** (`StepTypeValidation`): Verify outcomes

Step Actions

Predefined actions for common operations:

```
const (
    StepActionAnalyzeCode    StepAction = "analyze_code"
    StepActionGenerateCode   StepAction = "generate_code"
    StepActionExecuteCommand StepAction = "execute_command"
    StepActionRunTests       StepAction = "run_tests"
    StepActionLintCode       StepAction = "lint_code"
    StepActionBuildProject   StepAction = "build_project"
)
```

DAG-Based Execution

Steps execute based on dependency resolution:

```
func (e *Executor) areDependenciesCompleted(workflow *Workflow,
        step *Step) bool {
    for _, depID := range step.Dependencies {
        depCompleted := false
        for _, s := range workflow.Steps {
            if s.ID == depID && s.Status == StepStatusCompleted {
                depCompleted = true
                break
            }
        }
```

```
            if !depCompleted {
                return false
            }
        }
        return true
    }
```

LLM Integration

The executor integrates with LLM providers for intelligent operations:

Analysis with LLM: - Gathers project context (files, dependencies, entry points) - Builds structured prompts with project information - Uses low temperature (0.2) for analytical accuracy

Generation with LLM: - Provides project context and requirements - Uses slightly higher temperature (0.3) for creativity - Falls back to templates when LLM unavailable

Template Fallback: When LLM is unavailable, generates language-specific templates for Go, Node.js, Python, and Rust.

Security Model

Command execution includes multiple security measures:

```
func isDangerousCommand(cmd string) bool {
    dangerousPrefixes := []string{
        "rm ", "dd ", "mkfs", "fdisk", "parted",
        "wipefs", "shred", "shutdown", "reboot",
        // ... more dangerous prefixes
    }

    dangerousPatterns := []string{
        "rm -rf /", "--no-preserve-root",
        "| sh", "| bash", "`rm", "$(rm",
        // ... more dangerous patterns
    }
    // Pattern matching logic
}
```

Workflow Types

Predefined workflow templates:

Planning Workflow: 1. Analyze requirements 2. Generate architecture (depends on analysis)

Building Workflow: 1. Setup environment 2. Compile code (depends on setup)

Testing Workflow: 1. Run unit tests 2. Run integration tests (depends on unit tests)

Refactoring Workflow: 1. Analyze codebase 2. Refactor code (depends on analysis)

Metrics Collection

```
type ExecutionMetrics struct {
    WorkflowsStarted int64
    WorkflowsSuccess int64
    WorkflowsFailed  int64
    StepsExecuted    int64
    StepsFailed      int64
    TotalDuration    time.Duration
    LLMCalls         int64
    LLMTokensUsed    int64
}
```

Autonomy Modes

The workflow system supports different autonomy levels through the `autonomy` subpackage: - Full autonomy (auto-approve all actions) - Supervised (require approval for certain actions) - Manual (require approval for all actions)

Consequences

Positive

1. **Flexibility**: DAG model supports complex dependency graphs
2. **Intelligence**: LLM integration enables adaptive execution
3. **Security**: Command validation prevents injection attacks
4. **Observability**: Comprehensive metrics and status tracking
5. **Graceful Degradation**: Template fallback when LLM unavailable
6. **Multi-language**: Built-in support for common languages
7. **Extensibility**: New step types and actions easily added
8. **Concurrency**: Independent steps can execute in parallel

Negative

1. **Complexity**: DAG execution more complex than linear pipelines
2. **LLM Dependency**: Optimal results require LLM availability
3. **Template Limitations**: Fallback templates are basic
4. **Security Trade-offs**: Some legitimate commands may be blocked

Neutral

1. **Learning Curve**: Teams need to understand DAG model
2. **Configuration**: Workflow definitions require careful design

Alternatives Considered

Alternative 1: Linear Pipeline Model

Description: Execute steps in a fixed linear order.

Pros: - Simple to understand - Predictable execution - Easy to debug - Straightforward implementation

Cons: - No parallel execution - Rigid structure - Unnecessary waiting for independent steps - Cannot model complex dependencies

Why Rejected: Real development workflows have complex dependencies. Linear execution would be inefficient and inflexible.

Alternative 2: Event-Driven Model

Description: Steps trigger based on events and message passing.

Pros: - Highly decoupled - Natural parallelism - Reactive to changes - Good for streaming workflows

Cons: - Harder to reason about execution order - Complex error handling - Debugging challenges - Eventual consistency issues

Why Rejected: Development workflows need deterministic execution for debugging and reproducibility.

Alternative 3: State Machine Model

Description: Model workflows as state machines with transitions.

Pros: - Well-defined states - Clear transitions - Easy to visualize - Good for approval workflows

Cons: - Limited parallelism - Complex for branching workflows - Explosion of states for complex workflows - Not natural for task dependencies

Why Rejected: Development workflows are naturally graphs, not state machines. Dependencies are on tasks, not states.

Alternative 4: Container-Based Execution (Tekton/Argo)

Description: Use Kubernetes-native workflow engines.

Pros: - Mature tooling - Container isolation - Resource management - Community ecosystem

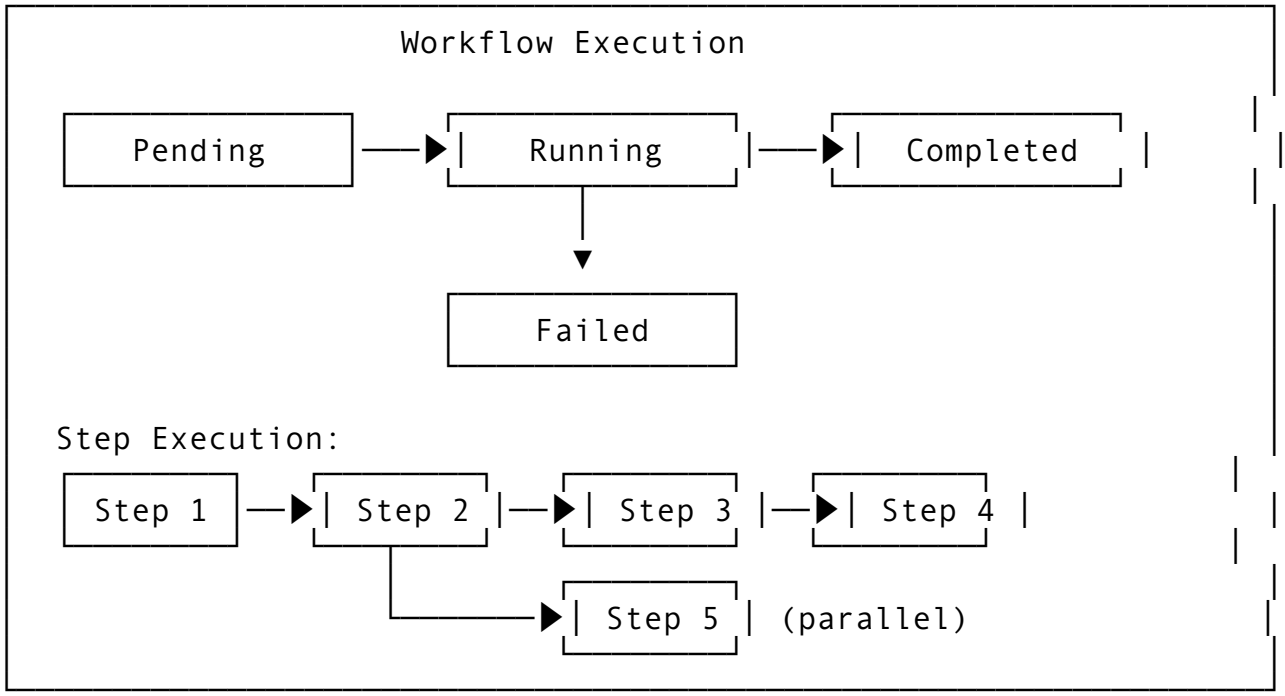
Cons: - Kubernetes requirement - Higher latency for simple tasks - Operational complexity - Overkill for many use cases

Why Rejected: Many HelixCode deployments don’t have Kubernetes. SSH-based execution is more universal and lower latency.

Implementation Notes

- Executor implementation in `internal/workflow/executor.go`
- Autonomy modes in `internal/workflow/autonomy/`
- Plan mode for planning-specific workflows in `internal/workflow/planmode/`
- Snapshots for state preservation in `internal/workflow/snapshots/`
- Command security validation is strict by default

Workflow Execution Flow



Related Decisions

- ADR-001: LLM Provider Interface (workflows use LLM for generation/analysis)
- ADR-002: Distributed Worker Architecture (workflows execute on workers)
- ADR-007: Test Framework Architecture (testing workflows)

References

- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/workflow/workflow.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/workflow/executor.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/workflow/autonomy/`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/workflow/planmode/`